

# Universidad de Costa Rica

## INGENIERÍA ELÉCTRICA



# INTELIGENCIA ARTIFICIAL APLICADA A LA INGENIERÍA

*Proyecto: clasificación de contaminaciones*

Autor:  
Cristhian Rojas Álvarez

Carnet: C16760

Mayo 2026

# 1. Objetivos

## 1.1. Objetivo general

Desarrollar un sistema capaz de clasificar automáticamente la presencia de contaminantes (granos de arroz) frente a objetos de control (clips) en un entorno de producción simulado.

## 1.2. Objetivos específicos

- Implementar un flujo de preprocesamiento digital de imágenes para binarizar y estandarizar fotografías capturadas en condiciones variables.
- Consolidar y curar un conjunto de datos colaborativo, aplicando filtros morfológicos para reducir el ruido visual y mejorar la calidad de la información.
- Entrenar y evaluar diversos algoritmos de aprendizaje automático clásico (SVM, KNN, Naive Bayes, Árboles de Decisión) para seleccionar el modelo con mayor capacidad de generalización.

# 2. Introducción

Imagina una banda transportadora en una fábrica que se mueve a toda velocidad. Pasan miles de productos frente a tus ojos y, sinceramente, es humanamente imposible notar si cae una basurita o un pequeño contaminante en medio de tanto movimiento. Ahí es donde entra nuestro proyecto.

Lo que se hizo fue crear una especie de clasificador. Básicamente, una cámara toma fotos de la superficie de trabajo y, mediante un proceso digital, limpiamos la imagen hasta que solo quedan siluetas claras (en blanco y negro). Después, entrenamos a un programa para que aprenda a diferenciar qué es parte del producto y qué es un "intruso".

# 3. Adquisición y Estandarización de Datos

La fase de recolección de datos se llevó a cabo de manera colaborativa, involucrando la participación de 10 estudiantes. Este enfoque introdujo una variabilidad significativa en el conjunto de datos inicial, debido a factores como:

- Diferencias en la resolución y óptica de las cámaras de los dispositivos móviles.
- Variaciones críticas en las condiciones de iluminación (luz natural vs. luz artificial).
- Distintas perspectivas y distancias de captura respecto a la superficie de trabajo.

Para mitigar este sesgo y asegurar la convergencia de los modelos de aprendizaje, se implementó un proceso de estandarización riguroso. Cada imagen fue redimensionada a una resolución fija de  $128 \times 128$  píxeles. Posteriormente, se aplicó una técnica de binarización para simplificar la

representación visual a un espacio booleano, facilitando la distinción entre el fondo (hoja blanca) y los objetos (arroz o clips).

Desde el punto de vista computacional, este proceso transforma una matriz bidimensional en un vector de características unidimensional. El número total de entradas para el modelo se define por el producto de las dimensiones de la imagen:

$$D = 128 \times 128 = 16,384 \text{ características (píxeles)} \quad (1)$$

Esta vectorización permite que el algoritmo de *Support Vector Machine* (SVM) procese la imagen como un punto en un espacio de alta dimensionalidad, donde la columna final representa la etiqueta de clase  $y \in \{0, 1\}$ .

## 4. Pipeline de Preprocesamiento y Limpieza Morfológica

Una de las etapas críticas del proyecto fue el tratamiento de las imágenes binarizadas mediante técnicas de visión artificial utilizando la librería *OpenCV*. El objetivo primordial fue eliminar el ruido impulsivo y las imperfecciones derivadas del proceso de umbralización (*thresholding*) [1].

Para mejorar la calidad de las muestras, se aplicaron dos operaciones morfológicas fundamentales descritas por González y Woods [1]:

1. **Apertura (Opening):** Definida como una erosión seguida de una dilatación ( $A \circ B = (A \ominus B) \oplus B$ ). Esta operación permitió eliminar pequeños objetos aislados (ruido tipo "sal") y protuberancias delgadas que no corresponden a la estructura geométrica del arroz.
2. **Cierre (Closing):** Definida como una dilatación seguida de una erosión ( $A \bullet B = (A \oplus B) \ominus B$ ). Se utilizó para cerrar pequeñas brechas o "huecos" dentro de los objetos binarizados, asegurando que cada contaminante se represente como una masa compacta.

El análisis de este *pipeline* revela que, sin esta etapa de curación, el modelo de aprendizaje automático sería propenso al *overfitting* sobre el ruido ambiental. Al eliminar artefactos visuales como sombras mal procesadas o motas de polvo en la superficie de captura, obligamos al clasificador a centrar su aprendizaje en la morfología (forma y tamaño) real de los objetos, aumentando así la robustez del sistema ante datos provenientes de diferentes entornos de captura.

## 5. Procesamiento, Integración y Curación de Datos

La calidad del modelo de aprendizaje depende directamente de la integridad del conjunto de datos. Dado que las muestras fueron recolectadas de forma descentralizada por un grupo de 10 colaboradores, se desarrolló un flujo de trabajo automatizado para la consolidación y limpieza de la información.

## 5.1. Integración de Datasets (Pipeline de Concatenación)

Se implementó un algoritmo en Python utilizando las librerías `pandas`, `glob` y `os` para unificar los archivos fuente en un único *dataset* maestro. El proceso siguió una lógica de validación estricta para asegurar la interoperabilidad de los datos:

- **Detección Dinámica de Formato:** El sistema analiza la primera línea de cada archivo para identificar el separador utilizado (; o ), permitiendo la lectura correcta de archivos generados en diferentes configuraciones regionales.
- **Normalización de Encabezados:** Se implementó una lógica de detección de metadatos que identifica si un archivo contiene etiquetas de columna (basándose en el tipo de dato de la primera celda o en el conteo de filas). En caso afirmativo, el encabezado es removido para conservar únicamente los vectores numéricos.
- **Validación Dimensional:** Cada archivo es sometido a una prueba de integridad para confirmar que posee exactamente el número de columnas esperado para una imagen de  $128 \times 128$  píxeles más su etiqueta:

$$C_{total} = (128 \times 128) + 1 = 16,385 \text{ columnas} \quad (2)$$

El resultado de esta etapa es el archivo `dataset_matriz_concatenada.csv`, que consolida la totalidad de las capturas del grupo.

## 5.2. Curación y Filtrado de Outliers

Para evitar que el modelo aprenda de capturas defectuosas, se aplicó un filtro basado en la densidad de píxeles. Dado que el fondo (hoja blanca) predomina sobre los objetos, se calculó el promedio de intensidad  $\mu$  de la imagen binarizada, aplicando los siguientes criterios de exclusión:

- **Muestras Saturadas o Vacías** ( $\mu > 0,99$ ): Se eliminaron imágenes donde el ruido de iluminación eliminó las siluetas o donde no se detectó ningún objeto, dejando la matriz casi totalmente blanca.
- **Muestras con Oclusión o Error de Segmentación** ( $\mu < 0,85$ ): Se descartaron capturas con exceso de sombras o ruido de fondo, donde el área oscura representaba una proporción irreal para el tamaño de un grano de arroz o un clip.

## 5.3. Refinamiento Espacial (Limpieza Morfológica)

Posterior al filtrado estadístico, cada muestra fue reconstruida a su forma original de  $128 \times 128$  para aplicar técnicas de procesamiento digital de imágenes mediante la biblioteca `OpenCV`. Se utilizó un elemento estructurante (*kernel*) de  $3 \times 3$  píxeles para ejecutar las siguientes operaciones:

1. **Apertura Morfológica (*Opening*):** Utilizada para la remoción de ruido impulsivo tipo "sal", eliminando pequeños artefactos blancos que no forman parte de la geometría del arroz.

2. **Cierre Morfológico (*Closing*)**: Aplicada para rellenar discontinuidades dentro de los objetos segmentados, asegurando que el área del contaminante sea compacta y libre de oquedades.

Finalmente, las imágenes procesadas fueron *“planadas”* (*flattened*) nuevamente para generar el archivo `dataset_grupal_super_limpio.csv`, el cual constituye la entrada oficial para la fase de entrenamiento.

## 6. Desarrollo y Evaluación de Modelos de Aprendizaje Automático

Para la fase de clasificación, se implementó un flujo de trabajo basado en la biblioteca *scikit-learn*, comparando cinco arquitecturas de aprendizaje supervisado. El objetivo fue identificar el modelo con mayor capacidad predictiva sobre datos no observados, utilizando una metodología de optimización de hiperparámetros.

### 6.1. Partición del Dataset y Validación Cruzada

El conjunto de datos final, denominado `dataset_grupal_super_limpio.csv`, fue dividido mediante una técnica de *hold-out* con una proporción de 80 % para el entrenamiento ( $X_{train}$ ) y 20 % para la prueba ( $X_{test}$ ).

Para evitar el sobreajuste (*overfitting*) y asegurar la robustez de los resultados, se integró la técnica de **Validación Cruzada de K-Pliegues** ( $k = 5$ ) dentro de una búsqueda de rejilla (*Grid Search*). Esto garantiza que cada configuración de hiperparámetros se evalúe cinco veces sobre diferentes subconjuntos de los datos de entrenamiento antes de seleccionar la mejor combinación.

### 6.2. Configuración del Espacio de Búsqueda (Grid Search)

Se definieron rejillas de parámetros (*param\_grids*) específicas para cada algoritmo, permitiendo al sistema explorar sistemáticamente las configuraciones óptimas:

#### 6.2.1. Árbol de Decisión (DecisionTreeClassifier)

Se exploró la profundidad del árbol para controlar la complejidad del modelo y el criterio de impureza para las ramificaciones:

- **Criterios**: Gini y Entropía.
- **Profundidad máxima** (*max\_depth*): {5, 10, 20, None}.
- **División mínima** (*min\_samples\_split*): {2, 5, 10}.

### 6.2.2. K-Vecinos más Cercanos (KNeighborsClassifier)

Se analizó la sensibilidad del modelo al número de vecinos y la influencia de la distancia en la votación:

- **Vecinos ( $k$ ):**  $\{3, 5, 7, 9\}$ .
- **Pesos ( $weights$ ):** *Uniform* (voto igualitario) y *Distance* (voto ponderado por la cercanía).

### 6.2.3. Máquina de Soporte Vectorial (SVC)

Se evaluaron núcleos lineales y no lineales, junto con el parámetro de regularización  $C$  que controla el compromiso entre el margen y el error de clasificación:

- **Regularización ( $C$ ):**  $\{0.1, 1, 10\}$ .
- **Núcleos ( $kernels$ ):** *Linear* y *RBF* (Radial Basis Function).

### 6.2.4. Bosque Aleatorio (RandomForestClassifier)

Se implementó un ensamble de árboles para mejorar la estabilidad del sistema, variando la cantidad de estimadores y la profundidad:

- **Estimadores ( $n$ ):**  $\{50, 100\}$ .
- **Profundidad máxima:**  $\{10, 20, \text{None}\}$ .
- **Criterios:** Gini y Entropía.

### 6.2.5. 5. Naive Bayes Gaussiano (Baseline)

Se utilizó el modelo `GaussianNB` como línea base de comparación. Al no requerir ajuste de hiperparámetros complejos, permite establecer un umbral mínimo de desempeño esperado para el dataset binarizado.

## 6.3. Criterio de Selección y Persistencia

La evaluación final se realizó sobre el conjunto de prueba ( $X_{test}$ ), el cual no participó en la etapa de *GridSearch*. El criterio de selección del "Modelo Ganador" se basó estrictamente en la métrica de **Exactitud (*Accuracy*)**:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

### 6.3.1. Análisis del Modelo Seleccionado (SVM)

Como se observa en la Figura 1 (Matriz de Confusión), el modelo SVM alcanzó una exactitud del 0.70. Al desglosar el comportamiento del clasificador sobre las 37 muestras de prueba, se identificaron los siguientes puntos clave:

- **Verdaderos Negativos (12):** El sistema identificó correctamente 12 clips.
- **Verdaderos Positivos (14):** Se clasificaron acertadamente 14 granos de arroz.
- **Falsos Positivos (6):** El modelo confundió 6 clips con arroz. Este es un punto crítico, ya que implicaría que un contaminante pasaría por la línea de producción sin ser detectado.
- **Falsos Negativos (5):** Se marcaron 5 granos de arroz como contaminantes, lo que en un entorno real representaría un desperdicio innecesario de producto.

Tras la competencia de modelos, el algoritmo con el puntaje más alto fue exportado utilizando la biblioteca `joblib` bajo el nombre `C16760_Cristhian_Rojas_Alvarez.joblib`. Este archivo contiene el estimador final con sus hiperparámetros optimizados, listo para la fase de inferencia industrial.

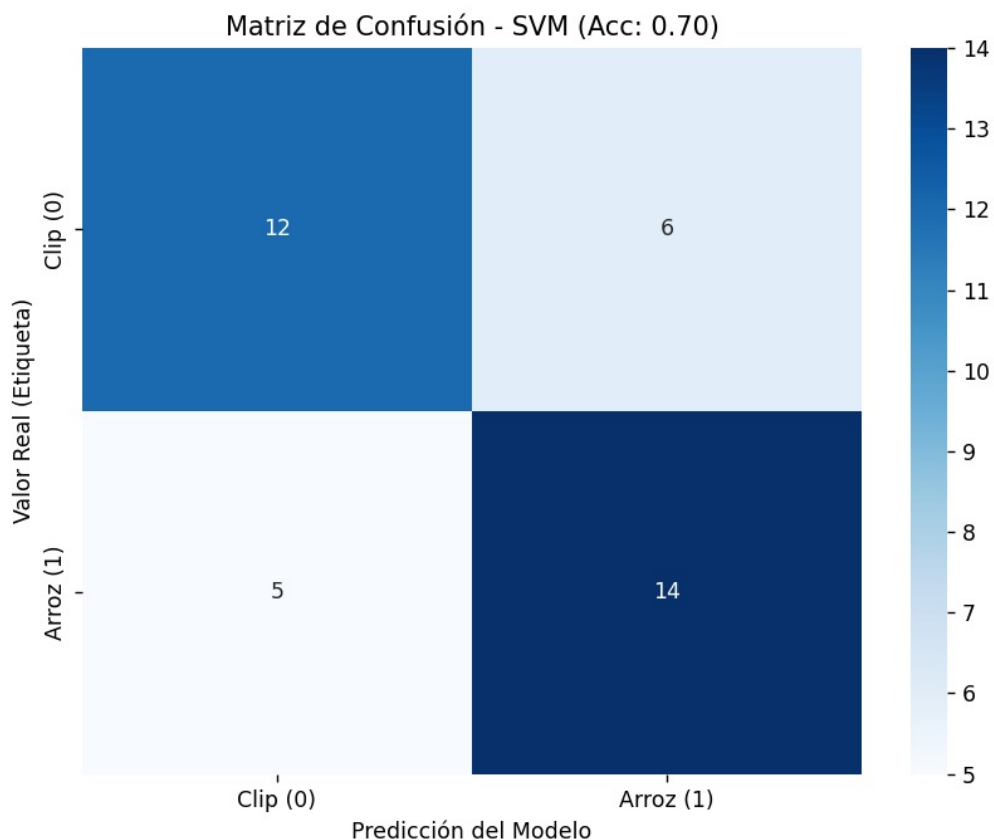


Figura 1: Matriz de confusión para el modelo SVM.

## 7. Conclusiones

- Se desarrolló con éxito un sistema capaz de clasificar contaminantes en una línea de producción simulada, alcanzando una exactitud del 70
- Las operaciones morfológicas de apertura y cierre resultaron fundamentales debido a la naturaleza de las capturas. Como se observa en las muestras iniciales, las sombras y los reflejos de luz en los clips generaban "huecos" en la binarización; el uso de filtros morfológicos permitió compactar estas masas y eliminar el ruido periférico, mejorando la calidad del vector de características.
- La heterogeneidad de los datos, producto de la recolección con 10 dispositivos diferentes, confirmó que el preprocesamiento de estandarización ( $128 \times 128$ ) y el filtrado estadístico de píxeles son pasos obligatorios para evitar el sobreajuste. El filtrado de muestras con  $\mu$  fuera de los rangos esperados aseguró que el modelo solo entrenara con imágenes donde la silueta del objeto fuera clara y representativa.
- Finalmente, se demostró que para este tipo de tareas de inspección visual, los modelos de aprendizaje clásico ofrecen un punto de partida sólido y eficiente. No obstante, para reducir el margen de error del 30 % restante, se recomienda en futuras iteraciones el uso de técnicas de aumento de datos (*Data Augmentation*) para compensar las variaciones de iluminación que aún afectan la precisión del sistema.

## Referencias

- [1] González, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.
- [2] Bradski, G. (2000). The OpenCV Library. *Dr. Dobbs's Journal of Software Tools*.
- [3] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [4] Joblib Development Team. (2020). *Joblib: Running Python functions as pipeline jobs*. <https://joblib.readthedocs.io/>
- [5] Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297.